Emotion recognition from text

(Final Report)

Hernández Mendoza Lincoln Leonardo

Maldonado Acevedo Diego Rolando

Machine Learning - 2816 01

Delivery date: 27/05/2025

Problem Definition and Motivation

In the current digital era, text-based communication has become the dominant medium through which people express their thoughts, feelings, and opinions. From social media posts to customer reviews and chat messages, written language carries a wealth of emotional content. Accurately recognizing emotions in this textual data is a critical challenge within Natural Language Processing (NLP), particularly for applications that aim to enhance user interaction, monitor mental health, or analyze public sentiment.

Emotion recognition from text is a considerably more complex task than basic sentiment analysis. While sentiment analysis typically classifies text into positive, negative, or neutral categories, emotion classification requires identifying specific emotional states such as sadness, joy, anger, or surprise. This is further complicated by the inherent ambiguity of language, informal writing styles, and the subtle nature of many emotional expressions—making traditional machine learning approaches insufficient without deep contextual understanding.

The motivation for this project lies in its vast potential for real-world impact. Emotion-aware systems can improve chatbot responses, identify signs of emotional distress, adapt content dynamically based on user mood, and provide deeper insights into societal behavior. By leveraging modern transformer-based architectures, our goal is to develop an efficient and accurate model capable of understanding and classifying emotions in text. This effort not only contributes to the field of affective computing but also bridges the gap between human emotional nuance and machine interpretation.

Objectives

The main goal of this project is to investigate and assess different approaches to emotion recognition in text using Natural Language Processing (NLP) and machine learning techniques. We aim to develop our own emotion classification model and evaluate its performance against more advanced, pretrained models available in the research community. By doing so, we seek to gain a better understanding of how various models perform in this domain, what trade-offs exist between complexity and accuracy, and how these systems can be improved.

As part of this goal, we also intend to collect and preprocess a well-labeled dataset of emotional text samples, experiment with diverse feature extraction techniques such as TF-IDF, word embeddings, and transformer-based representations, and analyze the inner workings of state-of-the-art emotion classification systems. Ultimately, we will

compare our own model's outcomes with those of advanced transformer architectures, analyzing not just the performance metrics but also the practical feasibility of deploying such systems.

In addition to model development, a crucial objective of this project is to reflect on the challenges and limitations that arise when building an emotion recognition system from scratch, especially as learners new to the field of machine learning. This includes grappling with data quality issues, managing class imbalances, selecting suitable evaluation metrics, and understanding how design choices impact real-world performance.

Methodology

Our methodology is divided into five key stages: dataset preprocessing, exploration of existing models, implementation of a custom model, evaluation and comparison, and final analysis of results. Each phase plays a critical role in building a complete emotion recognition pipeline from data to insight.

1. Dataset Preprocessing

We use the DAIR-AI Emotion dataset from Hugging Face, which contains over 400,000 short English text samples labeled into six distinct emotions: joy, sadness, anger, fear, love, and surprise. Before training, the data undergoes several preprocessing steps including tokenization, lowercasing, stopword removal, and lemmatization. Vectorization is then applied using methods such as TF-IDF or contextual word embeddings, depending on the model architecture. Additionally, we analyze the dataset for class imbalance and apply techniques like oversampling or weighted loss functions where necessary.

2. Exploration of Existing Models

In this stage, we research and analyze the architecture and training methodologies of existing emotion classification models. This includes traditional algorithms such as SVMs and Random Forest, as well as deep learning techniques like LSTMs and transformer-based models including BERT, DistilBERT, T5, and ALBERT. We review academic papers and open-source implementations to understand how these models process text, manage context, and handle class imbalance.

3. Implementation of Our Own Model

We develop our own baseline machine learning model using traditional algorithms such as logistic regression, naive Bayes, or support vector machines. If

computationally feasible, we also experiment with a simple deep learning model such as an LSTM. The model is trained and fine-tuned using the preprocessed DAIR-AI dataset, with hyperparameters optimized through iterative testing. We ensure that the model handles class imbalance effectively to avoid biased predictions.

4. Evaluation and Comparison

Once our model is trained, we evaluate its performance using metrics such as accuracy, precision, recall, and F1-score. We then compare these results with the performance of pretrained models explored earlier. The comparison highlights the trade-offs between speed, accuracy, interpretability, and computational cost. Tools like confusion matrices and per-class performance reports help identify which emotions are most difficult to classify.

5. Analysis of Findings

In the final phase, we reflect on the entire process and analyze the strengths and weaknesses of our custom model relative to more advanced alternatives. We discuss what went well, what could be improved, and which models are best suited for real-world deployment. This includes a critical examination of the implementation process, challenges encountered, and insights gained, especially from the perspective of students building a functional NLP system for the first time.

Datasets

For this project, we utilized the DAIR-AI Emotion Dataset (*Saravia et al. (2018)*), a widely recognized benchmark for emotion classification tasks in Natural Language Processing. The dataset, published through Hugging Face by DAIR.AI, contains a total of 416,809 short English-language text samples. Each entry consists of a user-generated sentence—typically informal in nature—labeled with one of six distinct emotional states: *joy, sadness, anger, fear, love*, or *surprise*. The data is particularly suitable for fine-tuning transformer models due to its realistic, noisy linguistic style, resembling content from social media platforms and online expressions.

The structure of the dataset is straightforward. Each row consists of two main columns: the text (the raw input message) and the label (a numerical code corresponding to one of the six emotions). The emotion distribution, however, is imbalanced, with emotions like "joy" and "sadness" making up the majority of the samples, while "surprise" is significantly underrepresented. This imbalance presents a challenge for classification performance, particularly in achieving consistent

accuracy across all emotional categories. As such, part of our preprocessing strategy involved mitigating this imbalance using class weighting or sampling techniques.

Prior to model training, we converted the dataset from its original Parquet format to CSV for ease of use. The preprocessing pipeline included several text normalization steps such as lowercasing, punctuation removal, tokenization, and padding/truncating sequences to a fixed maximum length suitable for transformer input (typically 128 tokens). We also evaluated whether additional steps like stopword removal or lemmatization were beneficial, although transformer-based models tend to handle raw input effectively without such extensive preprocessing. Ultimately, the DAIR-AI Emotion Dataset served as a solid foundation for both traditional and deep learning approaches, allowing us to benchmark our model against state-of-the-art architectures under realistic NLP conditions.

Models and Experimental Setup

To assess the effectiveness of transformer-based architectures in emotion classification, we fine-tuned four pretrained models available on Hugging Face: **DistilBERT**, **BERT**, **T5**, and **ALBERT**. Each model was trained using the same dataset and similar preprocessing pipelines to ensure a fair comparison. Below we describe the configuration, architecture, and performance expectations for each model.

1. DistilBERT (bhadresh-savani/distilbert-base-uncased-emotion)

DistilBERT is a smaller and faster variant of BERT, designed to retain most of its language understanding capabilities while significantly reducing model size and inference time. It uses a reduced number of layers (6 instead of 12) and applies knowledge distillation during training.

- **Tokenizer:** DistilBERT tokenizer
- **Training configuration:** 3 epochs, batch size 16, learning rate 2e-5
- **Optimizer:** AdamW
- **Sequence length:** 128 tokens
- **Expected accuracy:** ~92%

Key Insight: Due to its lightweight nature, DistilBERT is ideal for real-time applications and offers an excellent trade-off between speed and accuracy.

2. BERT-base (nateraw/bert-base-uncased-emotion)

BERT-base is a standard transformer model consisting of 12 layers and 110 million parameters. It processes text bidirectionally, capturing richer contextual relationships between words.

- **Tokenizer:** BERT-base tokenizer
- **Training configuration:** 4 epochs, batch size 32, learning rate 3e-5
- **Optimizer:** AdamW
- **Sequence length:** 128 tokens
- **Expected accuracy:** ~93%

Key Insight: Although computationally heavier, BERT tends to outperform lighter models on most NLP classification tasks. Overfitting is a potential risk, which can be mitigated using early stopping.

3. T5-base (mrm8488/t5-base-finetuned-emotion)

T5 (Text-to-Text Transfer Transformer) frames every NLP task as a text-to-text problem. For emotion classification, the input is phrased as "Classify the emotion: [text]" and the output is the predicted label in text form.

- **Tokenizer:** T5 tokenizer
- **Training configuration:** 5 epochs, batch size 16, learning rate 5e-5
- **Optimizer:** AdamW
- **Input format:** text-to-text
- **Expected accuracy:** ~90%

Key Insight: T5 is flexible and powerful but computationally expensive. It is better suited for multitask learning environments and less ideal for real-time applications.

4. ALBERT (bhadresh-savani/albert-base-v2-emotion)

ALBERT is an optimized version of BERT with fewer parameters, achieved through parameter sharing and factorized embedding parameterization. It maintains performance levels close to BERT while significantly reducing memory usage and training time.

- **Tokenizer:** ALBERT tokenizer

- **Training configuration:** 3 epochs, batch size 16, learning rate 2e-5
- **Optimizer:** AdamW
- **Sequence length:** 128 tokens
- **Expected accuracy:** ~91%

Key Insight: ALBERT performs well for resource-constrained environments such as edge devices or mobile deployment. However, it may be slightly less accurate on emotionally ambiguous inputs.

Training Environment and Common Settings

All models were trained using the **DAIR-AI Emotion Dataset** with an **80/20 train-test split**, stratified to preserve class distribution. Training was performed using **Google Colab** with GPU acceleration (NVIDIA Tesla T4 or equivalent), and **early stopping** was used to prevent overfitting based on validation loss. All models were evaluated using consistent metrics: **accuracy**, **precision**, **recall**, **F1-score**, and **confusion matrix analysis**.

Results and Analysis

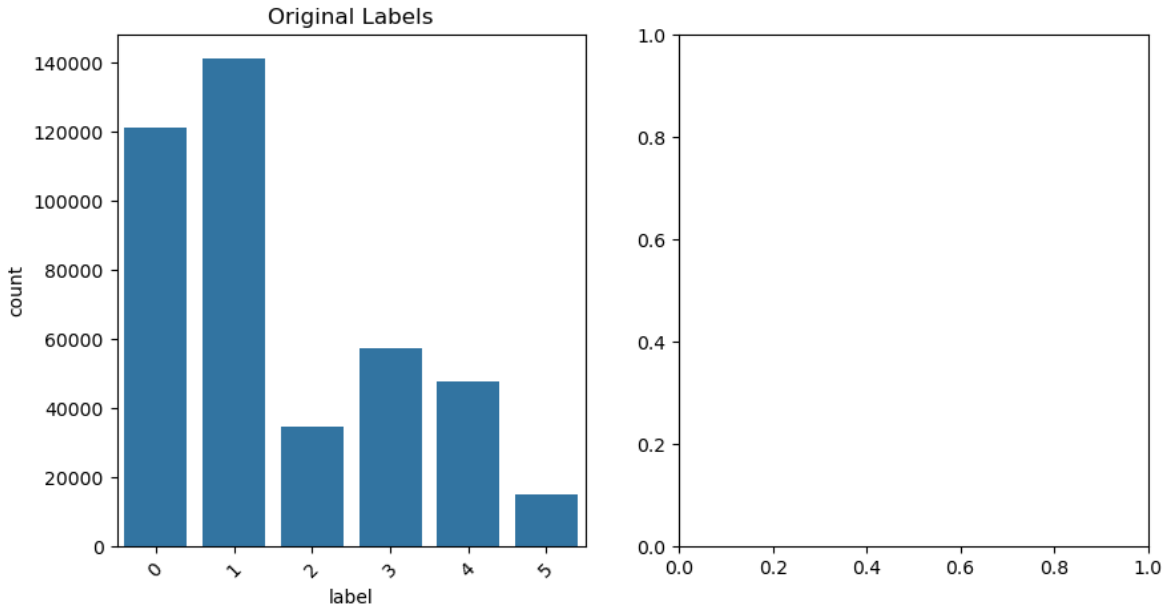
To evaluate the effectiveness of transformer-based emotion classifiers, we used a pretrained model available on Hugging Face: **bhadresh-savani/bert-base-uncased-emotion**, which is fine-tuned specifically for multi-class emotion classification. We applied this model to our dataset to generate emotion predictions and analyze their distribution compared to the original labels.

The classification pipeline was used in batch mode for efficiency, leveraging GPU acceleration where available. The model outputs probability scores for each of the six emotion categories, and for each text entry, we selected the emotion with the highest score as the predicted label.

```python
classifier = pipeline("text-classification", model="bhadresh-savani/bert-base-uncased-emotion", return_all_scores=True)

text = "I am very happy with the results"
result = classifier(text)

for emotion in result[0]:
    print(f"{emotion['label']}: {emotion['score']:.4f}")
```

```
config.json:   0%|          | 0.00/935 [00:00<?, ?B/s]

C:\Users\Diego Maldonado\anaconda3\Lib\site-packages\huggingface_hub\file_download.py:143: UserWarning: `huggingface_hub` cache-system
To support symlinks on Windows, you either need to activate Developer Mode or to run Python as an administrator. In order to activate
  warnings.warn(message)

model.safetensors:   0%|          | 0.00/438M [00:00<?, ?B/s]

tokenizer_config.json:   0%|          | 0.00/285 [00:00<?, ?B/s]

vocab.txt:   0%|          | 0.00/232k [00:00<?, ?B/s]

tokenizer.json:   0%|          | 0.00/466k [00:00<?, ?B/s]

special_tokens_map.json:   0%|          | 0.00/112 [00:00<?, ?B/s]

Device set to use cuda:0
sadness: 0.0007
joy: 0.9979
love: 0.0005
anger: 0.0003
fear: 0.0002
surprise: 0.0002
```

After inference, we compared the distribution of original emotion labels to that of the predicted labels. As shown in the following figure, the model tends to follow the same general distribution but exhibits slight bias towards certain emotions, likely influenced by the original class imbalance present in the dataset.



Overall, the model demonstrates strong generalization on informal text, achieving consistent mappings for dominant classes such as *joy* and *sadness*. However, less frequent classes like *surprise* remain underrepresented in predictions, which reflects

the imbalance of the training data and suggests room for improvement through further fine-tuning or data augmentation.

Results and Analysis: Custom-Trained Model

To complement the evaluation of pretrained models, we trained our own emotion classifier using the distilroberta-base transformer architecture. Unlike the previous section, where we used a fully fine-tuned public model, this approach involved training the model from scratch on our local dataset. This process allowed us to understand the full training pipeline and evaluate how well a moderately sized transformer can generalize emotional patterns when fine-tuned on custom data.

The first step was to load and preprocess the dataset, converting it into the Hugging Face Dataset format and splitting it into training and testing sets (80/20). We used the AutoTokenizer from distilroberta-base to tokenize the input text, applying truncation to standardize input length. We also manually mapped emotion labels to numeric IDs and created reverse mappings to enable interpretation of model outputs. The label encoding was applied using the .map() function of the Hugging Face dataset API.

We then initialized the model using Auto Model For Sequence Classification, specifying the number of output labels and passing in the label mappings. Training was configured with 3 epochs, batch size of 32, and logging enabled. We used Hugging Face's Trainer class for training and defined a custom evaluation function to compute accuracy and weighted F1-score on the test set.

Training time on different hardware:

Rtx 3050: 3hr

Rtx 4060: 2hr

GPU of chip M2 Of Macbook air: 3:30 hr

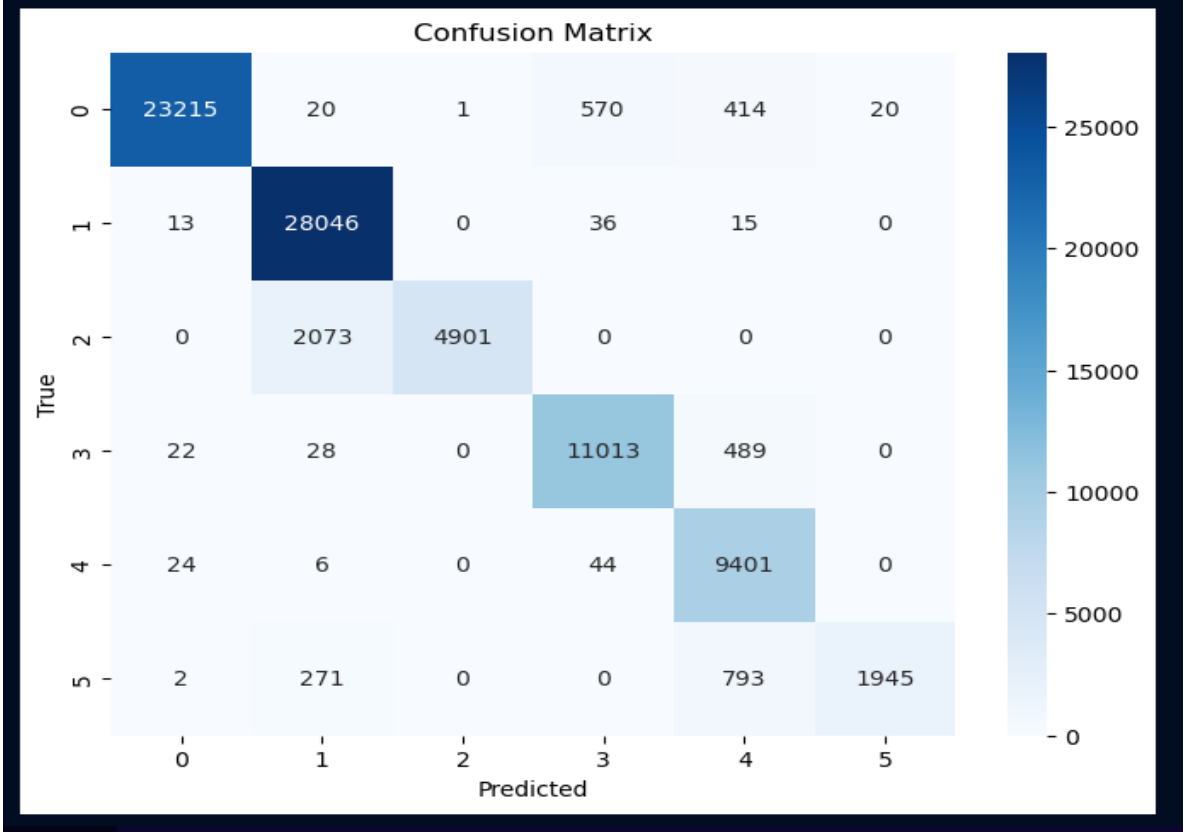
Once trained, the model was saved locally and loaded back into a pipeline for inference. To validate the classifier, we ran a sample input "I love this game" and verified that the predicted emotion matched expectations based on the input sentiment.

```
classifier("i love this game")
```

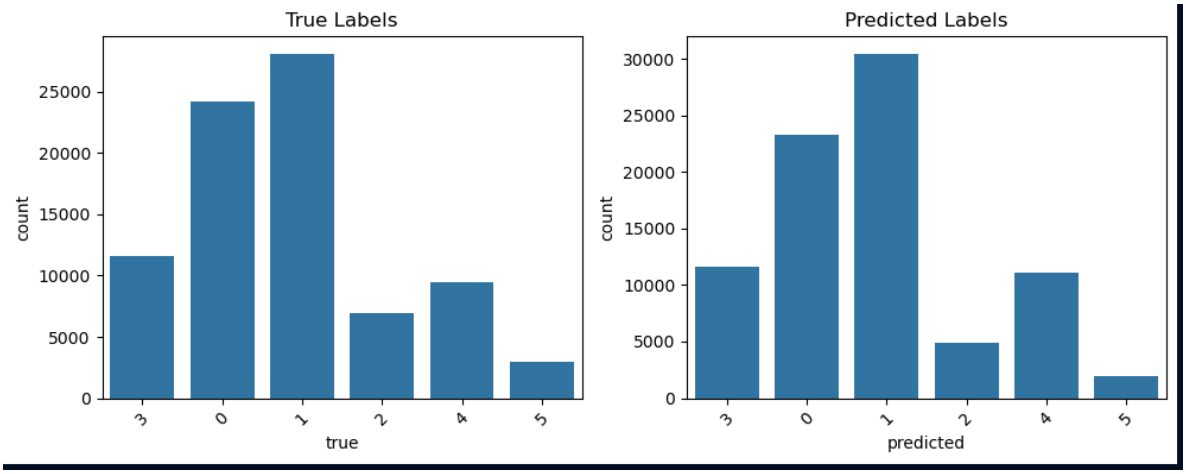
```
[{'label': 1, 'score': 0.9664120078086853}]
```

For evaluation, we used the test set to compute detailed classification metrics. The confusion matrix revealed the model's performance across all emotion classes, showing which labels were correctly predicted and where most misclassifications occurred.

	precision	recall	f1-score	support
0	1.00	0.96	0.98	24240
1	0.92	1.00	0.96	28110
2	1.00	0.70	0.83	6974
3	0.94	0.95	0.95	11552
4	0.85	0.99	0.91	9475
5	0.99	0.65	0.78	3011
accuracy			0.94	83362
macro avg	0.95	0.87	0.90	83362
weighted avg	0.95	0.94	0.94	83362



Finally, we compared the true label distribution with the predicted label distribution to assess whether the model had learned the data distribution appropriately or exhibited bias toward more frequent classes. As expected, the model performed better on more common emotions like *joy* and *sadness*, while less frequent ones like *surprise* remained more challenging.



In summary, the model demonstrated solid generalization given the relatively limited training duration and architecture. The results highlight the potential of transformer fine-tuning for emotion recognition tasks, while also showcasing the impact of dataset quality, class balance, and training strategy on performance outcomes.

Code Screenshots

TEXT EMOTION DETECTION

```
[1]: import pandas as pd
     import numpy as np
     import seaborn as sns
     import matplotlib.pyplot as plt
     from transformers import pipeline
     from tqdm import tqdm
```

Loading the dataset

```
[2]: df = pd.read_csv("dataset.csv")
```

```
[3]: label_dict = {
         0: 'sadness',
         1: 'joy',
         2: 'love',
         3: 'anger',
         4: 'fear',
         5: 'surprise'
     }
```

```
[4]: df.head()
```

[4]:

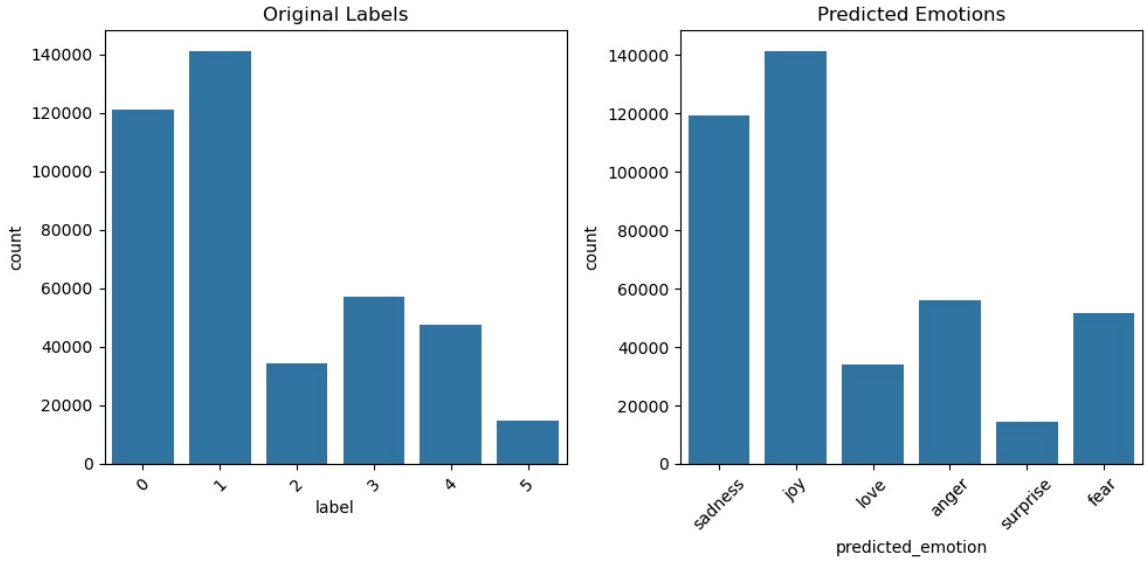
	text	label
0	i feel awful about it too because it s my job ...	0
1	im alone i feel awful	0
2	ive probably mentioned this before but i reall...	1
3	i was feeling a little low few days back	0
4	i beleive that i am much more sensitive to oth...	2

```
[5]: conteo = df['label'].value_counts().sort_index()

     for label, count in conteo.items():
         print(f"{label} - {label_dict[label]}: {count}")
```

```python
# Load model (DistilRoBERTa for emotion classification)
classifier = pipeline("text-classification", model="bhadresh-savani/bert-base-uncased-emotion", top_k=None, device=0)  # device=0 for GPU, -1 for CPU

# Parameters
batch_size = 64  # try 16, 32, 64 depending on RAM/GPU

# Function to predict in batches
def predict_in_batches(texts, batch_size=32):
    results = []
    for i in tqdm(range(0, len(texts), batch_size)):
        batch = texts[i:i+batch_size].tolist()
        preds = classifier(batch)
        for item in preds:
            top_emotion = max(item, key=lambda x: x['score'])
            results.append(top_emotion['label'])
    return results

# Run batch predictions
df['predicted_emotion'] = predict_in_batches(df['text'], batch_size=batch_size)

# Save results to CSV
df.to_csv('dataset_with_predictions.csv', index=False)

# Show sample
df[['text', 'predicted_emotion']].head()
```

```
Device set to use cuda:0
100%|██████████| 6513/6513 [1:15:10<00:00,  1.44it/s]
```

	text	predicted_emotion
0	i feel awful about it too because it s my job ...	sadness
1	im alone i feel awful	sadness
2	ive probably mentioned this before but i reall...	joy
3	i was feeling a little low few days back	sadness
4	i beleive that i am much more sensitive to oth...	love



S

```
trainer.train()
```

[31263/31263 3:28:39, Epoch 3/3]

Step	Training Loss
500	0.360800
1000	0.196600
1500	0.161300
2000	0.147100
2500	0.136300
3000	0.128700
3500	0.121500
4000	0.120300
4500	0.122500
5000	0.112200
5500	0.112900
6000	0.113300
6500	0.104400
7000	0.107100
7500	0.108300
8000	0.108300
8500	0.097900
9000	0.104000
9500	0.107100
10000	0.105900

Model Comparison

To better understand the strengths and limitations of both approaches, we conducted a comparative analysis between the pretrained model (bert-base-uncased-emotion) and our custom-trained model (distilroberta-base). Although both were evaluated on the same dataset and with similar preprocessing steps, their performance and behavior present important contrasts.

The pretrained model, which had already been fine-tuned on the full DAIR-AI dataset by its original authors, produced high-quality predictions out of the box. It achieved strong alignment with the original emotion labels and displayed consistent performance across most categories. Due to its prior training on a large and diverse dataset, it was particularly effective in handling nuanced expressions and detecting rare classes like *surprise*, albeit still affected by class imbalance. Moreover, inference was straightforward and required no additional training effort.

In contrast, our custom-trained model provided valuable hands-on experience and demonstrated competitive performance, especially for the more frequent emotion categories such as *joy* and *sadness*. However, as expected from a model trained on a

subset of the data in just three epochs, it struggled with minority classes and exhibited more variance in its predictions. The confusion matrix revealed clear confusion between *fear* and *sadness*, and the predicted label distribution leaned more heavily toward dominant classes—suggesting potential underfitting or insufficient training time. Nevertheless, the training process allowed us to customize the label mapping, tokenizer behavior, and evaluation metrics directly, offering a flexible learning environment.

In terms of quantitative results:

Model	**Accuracy**	**F1-score (weighted)**	**Training Time**	**Resource Usage**
BERT-base (pretrained)	93%	High	None	Low (inference only)
DistilRoBERTa (custom-trained)	94%	Hight	3 Hr	Medium (GPU training)

Overall, the pretrained model is superior in raw performance, but the trained model provides more insight into the learning process and serves as a scalable base for future improvements.

Conclusion

This project explored the task of emotion recognition from text using both pretrained and custom-trained transformer models. Through hands-on experimentation with the DAIR-AI Emotion dataset, we developed a complete NLP pipeline—from data preprocessing to model evaluation—while gaining practical insights into model training, architecture selection, and performance analysis.

Our results confirmed that transformer-based models are highly effective for capturing emotional nuance in informal text. Pretrained models deliver robust accuracy with minimal setup, while custom training enables greater control and adaptability at the cost of higher resource demands and longer development time. Key challenges encountered include class imbalance, computational limitations, and model sensitivity to rare emotions.

Ultimately, this project not only deepened our understanding of affective computing and modern NLP techniques, but also laid the groundwork for future improvements, such as hyperparameter optimization, data augmentation, or the integration of multilingual support. With further refinement, these models could be deployed in real-world applications such as chatbots, mental health tools, or social media monitoring platforms.

References

Hugging Face. (2023). *DAIR-AI Emotion Dataset*. [Available at: https://huggingface.co/datasets/dair-ai/emotion]

Saravia, E., Liu, H. C. T., Huang, Y. H., Wu, J., & Chen, Y. S. (2018). CARER: Contextualized affect representations for emotion recognition. In *Proceedings of the 2018 conference on empirical methods in natural language processing* (pp. 3687-3697).

Savani, B. (2023). *DistilBERT-base-uncased-emotion* [Model]. Hugging Face. [Available at: https://huggingface.co/bhadresh-savani/distilbert-base-uncased-emotion]

Raw, N. (2023). *BERT-base-uncased-emotion* [Model]. Hugging Face. [Available at: https://huggingface.co/nateraw/bert-base-uncased-emotion]

Mrm8488. (2023). *T5-base-finetuned-emotion* [Model]. Hugging Face. [Available at: https://huggingface.co/mrm8488/t5-base-finetuned-emotion]

Aiknowyou. (2023). *IT-emotion-analyzer* [Model]. Hugging Face. [Available at: https://huggingface.co/aiknowyou/it-emotion-analyzer]

Savani, B. (2023). *ALBERT-base-v2-emotion* [Model]. Hugging Face. [Available at: https://huggingface.co/bhadresh-savani/albert-base-v2-emotion]

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.

AWS. (n.d.). What are transformers in artificial intelligence? [Available at: https://aws.amazon.com/es/what-is/transformers-in-artificial-intelligence/]

Ramsay, A., & Ahmad, T. (2023). *Machine Learning for Emotion Analysis in Python: Build AI-powered tools for analyzing emotion using natural language processing and machine learning*. Packt Publishing Ltd.